SmartPlayer — Hex

Explicación Profunda de Cada Algoritmo y Heurística

Union-Find · MCTS · RAVE/AMAF · PUCT · Path-Connection · VC Solver · Dead Cell Pruning

Universidad de La Habana
Facultad de Matemática y Computación

Inteligencia Artificial
c-311

Melissa Maureen Sales Brito

1 Union-Find Incremental

Se implementa un DSU como estructura de datos eficiente para identificar amenaza/victoria inmediata. Empleando la idea de que no importa la forma exacta de la cadena sino a qué fichas conecta, para ello se agregan cuatro nodos virtuales a la estructura (VL = N^2, VR = N^2+1, VT = N^2+2, VB = N^2+3), se mantienen dos listas de longitud N^2+4 una para los padres y otra para las alturas; indexando cada nodo con la fórmula r * size + c.

1.1 Optimización con Path-Splitting

El clásico es utilizar path-compression que en una primera pasada sube hasta la raíz y en una segunda pasada vuelve a bajar haciendo que todos los nodos apunten directamente a la raíz; sin embargo nuestra elección de splitting es por motivo de eficiencia. En este caso se realiza una sola pasada donde, como nodo, en vez de apuntar a mi padre apunto a mi abuelo. No queda plano en un solo recorrido pero se aplana progresivamente en llamadas sucesivas.

1.2 Estructura de la Clase

```
Class IncrementalUF:

    def __init__(self, n):
    def find(self, a):
    def union(self, a, b):
    def clone(self):
```

2 Nodo Monte Carlo Tree Search (MCTS)

MCTS resuelve el problema de tomar decisiones en un espacio de estados muy grande donde explorar exhaustivamente no es la solución. En vez de analizar todas las posibilidades, simula muchas partidas aleatorias desde la posición actual y utiliza los resultados como estadísticas para estimar qué jugadas son probablemente buenas. Las jugadas que ganaron más simulaciones reciben más atención y las que recibieron más atención se exploran a mayor profundidad.

2.1 Etapas de MCTS

2.1.1 Selección

Baja por el árbol eligiendo el hijo más prometedor según la fórmula de puntuación combinada (Q_mcts, Q_amaf y PUCT). El camino resaltado indica la ruta elegida.

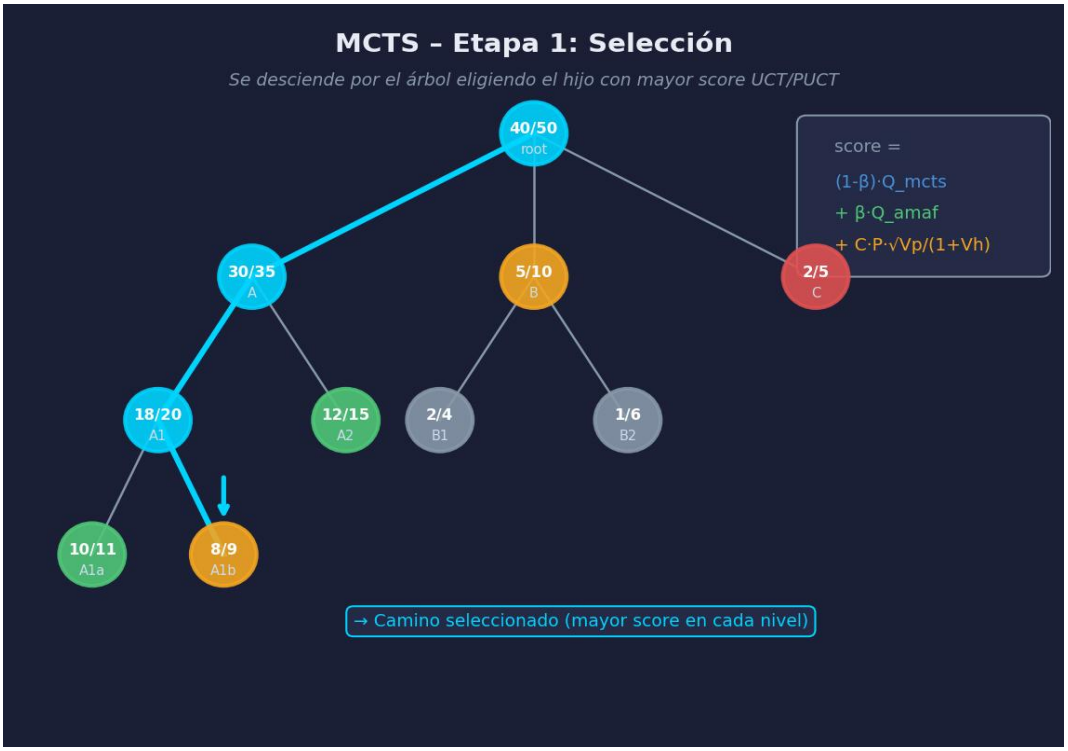

Figura 1 — Etapa de Selección: se desciende por el camino de mayor score

2.1.2 Expansión

Al llegar a un nodo con movidas no exploradas, se crea un nuevo hijo. El nuevo nodo recibe su prior de path-connection y comienza con visits = 0 wins = 0.

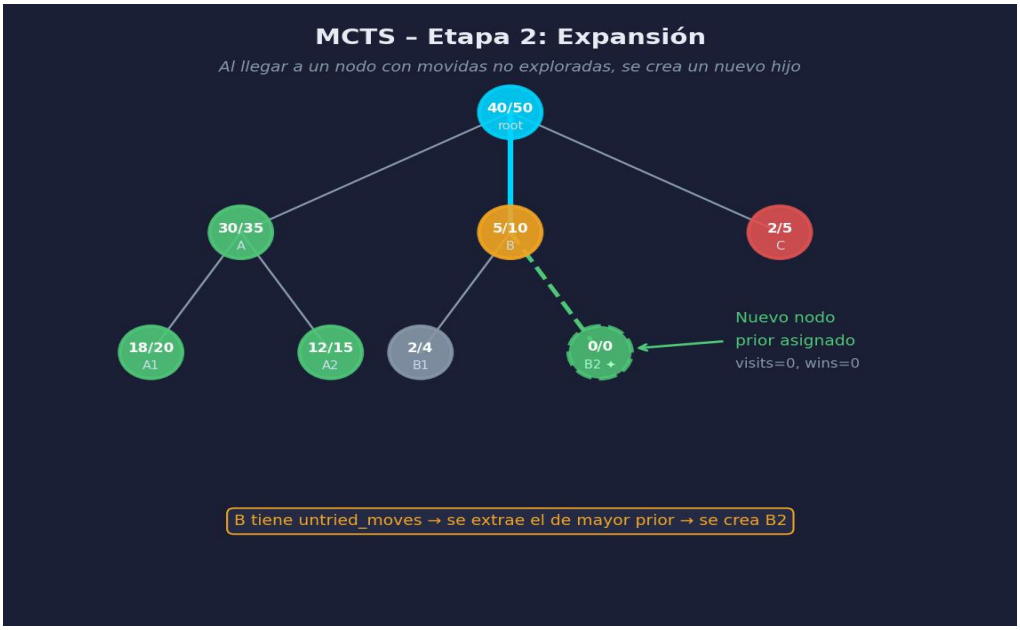

Figura 2 — Etapa de Expansión: se añade el nodo B2 con el mayor prior de untried_moves

2.1.3 Simulación (Rollout)

Desde el nuevo nodo, se juega hasta el final de manera rápida con orden semi-smart: score(r,c) = prior(r,c) + max_prior · U(0,1), produciendo ~70% de inteligencia y ~30% de aleatoriedad.

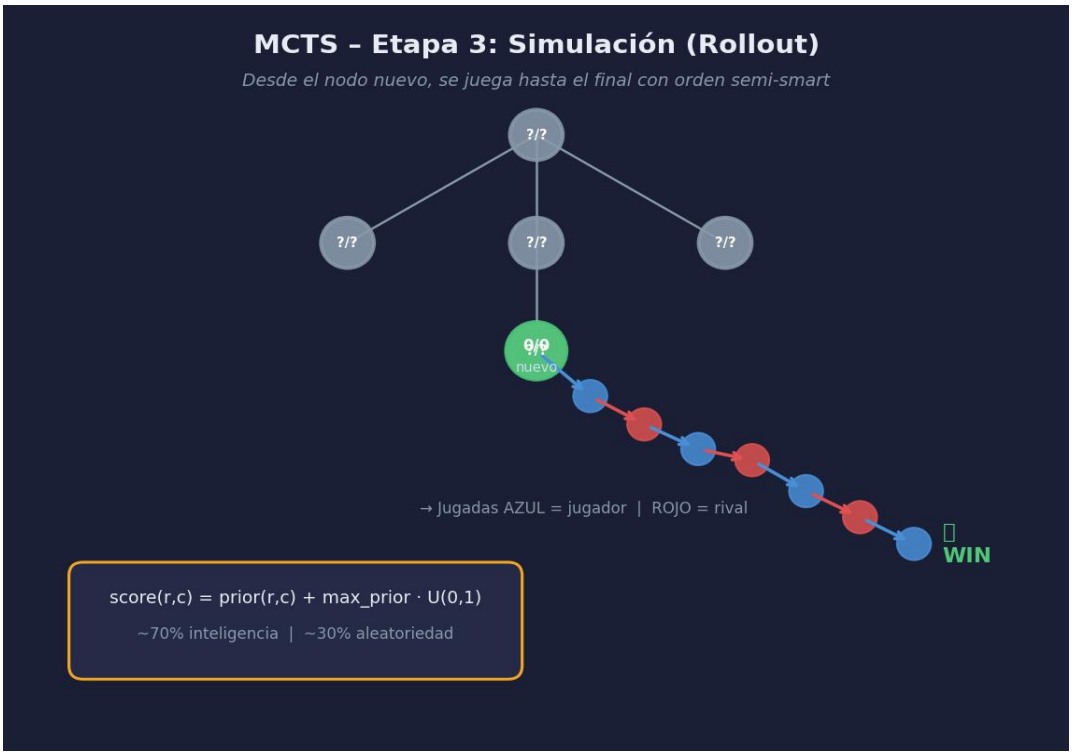

Figura 3 — Etapa de Simulación: rollout semi-smart hasta determinar el ganador

2.1.4 Retropropagación

El resultado WIN/LOSS sube por el árbol actualizando visits y wins en cada nodo visitado. Simultáneamente se actualizan los diccionarios amaf_v y amaf_w del nodo padre para todas las jugadas que aparecieron en el rollout.

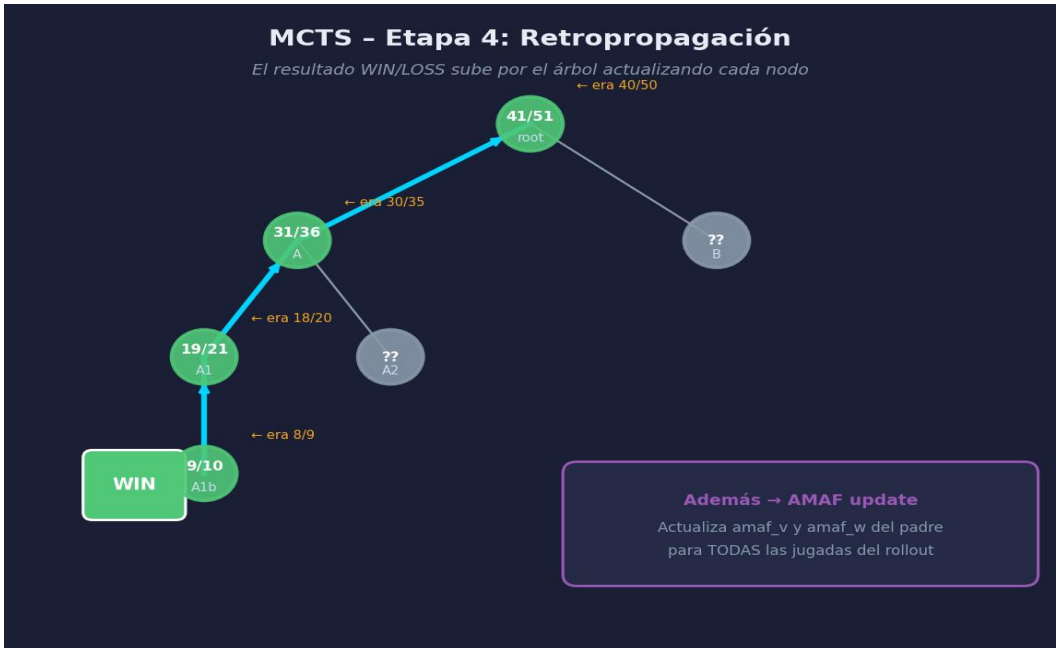

Figura 4 — Etapa de Retropropagación: el resultado asciende actualizando estadísticas

2.2 Estructura del Nodo

```
class MCTSNode:
    __slots__ = ['move','player','parent','children',
                 'untried_moves','visits','wins','prior','amaf_v','amaf_w']

    move   = move          # jugada (r,c) que llevó a este nodo
    player = player        # jugador que realizó el movimiento
    visits = 0             # veces que el algoritmo pasó por este nodo
    wins   = 0             # veces que terminó ganando
    children = []
    parent = parent
    prior                  # path-connection normalizado (fijo al crear)
    untried_moves = []     # lista (prior, move) ordenada ascendente
    amaf_v = {}            # estadística AMAF – visitas
    amaf_w = {}            # estadística AMAF – victorias
```

2.2.1 Sobre la Clase

El uso de __slots__ es una optimización de Python: en vez de un diccionario de atributos (el default), usa un array fijo. Con miles de nodos en el árbol, el ahorro de memoria es significativo y el acceso a atributos es más rápido.

Los campos visits y wins responden a: ¿Cuántas veces el algoritmo pasó por este nodo como primer movimiento real (no simulado)? y ¿Cuántas de esas veces terminó ganando (coincidiendo ganador-jugador)?

Los campos amaf_v y amaf_w constituyen diccionarios respecto a los hijos del nodo. Son la estadística más fuerte junto a prior. El campo prior almacena el valor del path-connection, asignado al crear el nodo y fijo durante toda la búsqueda. La lista untried_moves está ordenada ascendentemente por prior; el método pop() extrae el último elemento (el de mayor prior) dando O(1) para elegir la mejor movida no explorada.

2b Fórmula de Selección Combinada

El problema central de MCTS es decidir qué nodo visitar a continuación. Si siempre elegimos el que parece mejor (pura explotación), podemos perder jugadas mejores no descubiertas. Si exploramos demasiado uniformemente, desperdiciamos simulaciones en jugadas malas.

```
score = (1-β)·Q_mcts + β·Q_amaf + C·P·√V_padre / (1+V_hijo)
β = √(RAVE_K / (3·V_hijo + RAVE_K))   |   RAVE_K = 300, C = 0.8
```

3 Término Q_mcts

Q_mcts es el win-rate empírico del nodo: de todas las simulaciones que pasaron por este nodo, qué fracción terminó ganando.

```
Q_mcts = wins / visits
Fracción de simulaciones ganadas desde este nodo
```

3.1 El Problema de Q_mcts solo

Si solo usamos Q_mcts para seleccionar, el algoritmo colapsa en pura explotación: siempre elige el nodo con mayor win-rate. Pero al inicio de la búsqueda, los nodos tienen muy pocas visitas y sus Q_mcts son estadísticamente ruidosos.

Problema de explotación pura:

Nodo A: 1 visita, 1 victoria → Q_mcts = 1.0 (parece perfecto, pero es ruido)

Nodo B: 100 visitas, 80 victorias → Q_mcts = 0.8 (estadísticamente sólido)

Con explotación pura, jamás exploramos B aunque sea mucho mejor.

Necesitamos un bonus de exploración que decrezca con las visitas.

Por eso la fórmula suma un término de exploración (el PUCT) que empuja a visitar nodos poco explorados. Con más visitas, el bonus decrece y Q_mcts domina — exactamente lo que queremos.

4 Término Q_amaf

MCTS puro necesita muchas simulaciones para estimar la calidad de una jugada porque solo actualiza el nodo si esa jugada fue la primera en esa rama del árbol. Si el árbol tiene 100 movidas posibles, cada movida tarda mucho en acumular estadísticas.

Se basa en una heurística específica de Hex: si una jugada fue buena en una simulación que pasó por este nodo, probablemente también sería buena si la jugáramos directamente desde aquí. No importa en qué momento de la simulación apareció.

AMAF (All Moves As First) es la implementación de esa heurística. Cuando una simulación pasa por el nodo padre y termina con resultado R, actualiza las estadísticas AMAF del padre para todas las jugadas del jugador correspondiente que aparecieron en el rollout, no solo la primera.

El padre mantiene amaf_v y amaf_w para cada jugada de sus hijos:

amaf_v[mv] = cuántas simulaciones que pasaron por el padre tuvieron mv en algún punto del rollout

amaf_w[mv] = cuántas de esas simulaciones ganaron

Q_amaf = amaf_w[mv] / amaf_v[mv] si amaf_v > 0

Q_amaf = Q_mcts (fallback si no hay datos AMAF aún)

4.1 El Coeficiente Beta

¿Cuánto confiamos en Q_amaf vs Q_mcts? Al inicio (pocas visitas) Q_mcts es ruidoso y Q_amaf tiene más datos relativamente. Al final (muchas visitas) Q_mcts es más confiable porque es específico a este nodo.

```
β = √( RAVE_K / (3·V_hijo + RAVE_K) )
RAVE_K = 300.   Cuando V_hijo = 0: β = 1.   Cuando V_hijo → ∞: β → 0
```

5 PUCT — Exploración Guiada por Prior

5.1 UCT Clásico

El término de exploración clásico (UCT) es:

```
U_uct = C · √( log(V_padre) / V_hijo )
Todas las jugadas reciben el mismo bonus de exploración
```

UCT trata todas las jugadas como igualmente prometedoras hasta que las visita. En un tablero 11×11 con 121 posibles primeras jugadas, MCTS tarda muchas iteraciones en descubrir que el centro es mejor que una esquina.

5.2 PUCT

PUCT (Predictor + UCT, usado en AlphaZero) reemplaza el bonus uniforme por uno proporcional al prior de cada jugada:

```
U_puct = C · P · √V_padre / (1 + V_hijo)
P = prior de path-connection (normalizado). Jugadas críticas se exploran antes.
```

Si el prior de la jugada del centro es 0.15 y el de una esquina es 0.01, el centro recibe un bonus de exploración 15× mayor. MCTS lo visita antes, acumula estadísticas antes, y converge más rápido a la decisión correcta.

5.3 Prior

El prior P es la probabilidad normalizada que asigna el evaluador (path-connection) a cada jugada. La normalización garantiza que sumen 1, lo que hace que PUCT funcione como una distribución de probabilidad:

- Jugadas con prior alto: alta probabilidad de ser críticas según el evaluador. Reciben más exploración.
- Jugadas con prior bajo: poca probabilidad de ser críticas. Casi no se exploran salvo que Q_mcts lo justifique.

El prior es fijo al crear el nodo y no cambia durante la búsqueda. Solo Q_mcts y Q_amaf se actualizan con cada simulación. Esto permite que MCTS corrija errores del prior: si el path-connection asignó prior alto a una jugada mala, Q_mcts eventualmente bajará y el nodo será menos seleccionado.

6 Path-Connection Prior (Densidad de Caminos)

6.1 Dijkstra

Dijkstra puro: identifica el camino de distancia mínima desde un borde hasta cada celda y se puede obtener el prior de la celda inversamente proporcional a esta distancia. Pero dos celdas con la misma distancia mínima pueden tener importancias distintas.

Escenario	Problema de Dijkstra
Celda en un corredor único (todos los caminos la cruzan)	Distancia d → prior normal. Pero esta celda es CRÍTICA — sin ella no hay camino.
Celda con 3 caminos alternativos paralelos de la misma longitud	Distancia d → mismo prior. Pero es menos crítica porque hay rutas alternativas.

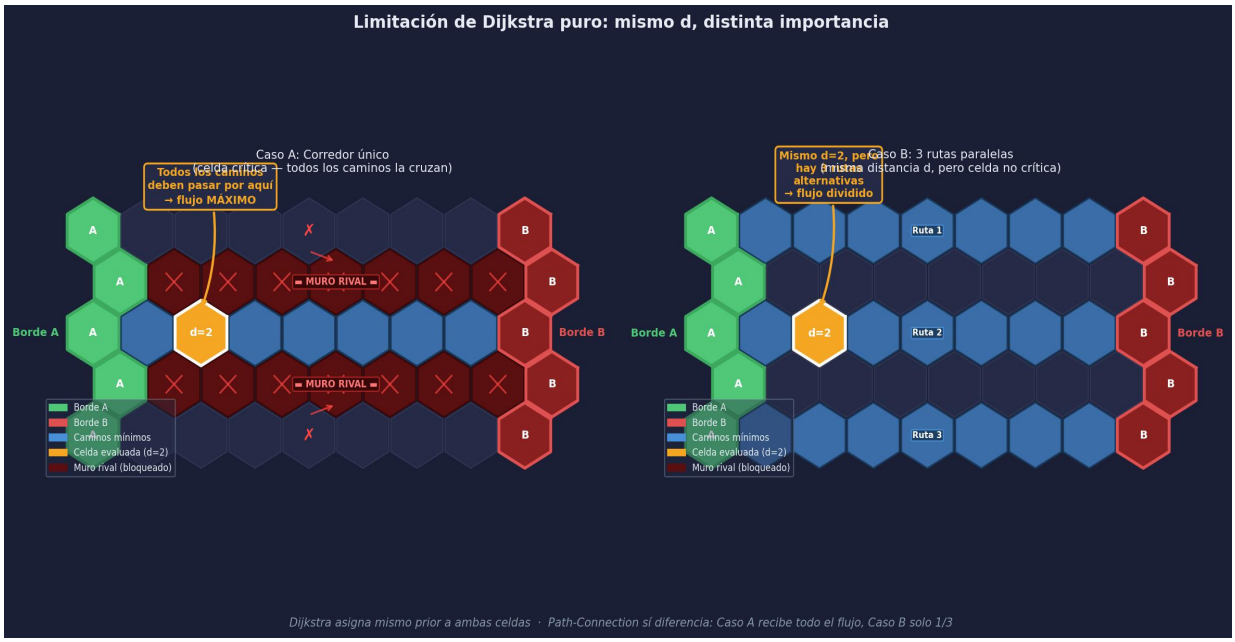

Figura 5 — Misma distancia d=2, distinta importancia real: Dijkstra no puede distinguirlas

6.1b DAG de Rutas Mínimas

```
def _dijkstra(self, board, size, player, forward):
    # Costos de arco:
    #   celda propia  → 0   (ya conectada, gratis)
    #   celda vacía   → 1   (hay que colocar una ficha)
    #   celda rival   → inf (bloqueada, ignorada)
    #
    # 'forward=True'  = desde el borde izquierdo/superior
    # 'forward=False' = desde el borde derecho/inferior
    #
    # dist[r][c] = mínimo número de fichas propias vacías necesarias
    #              para conectar el borde fuente hasta (r,c)
```

El DAG (grafo acíclico dirigido) de rutas mínimas se define implícitamente: el arco (u, v) pertenece al DAG si y solo si dist[v] == dist[u] + costo(u,v). Es decir, v está en el camino óptimo desde u. Este grafo tiene un orden topológico natural: simplemente ordena las celdas por dist[r][c] creciente. Una celda con distancia 3 nunca puede ser predecesora de una de distancia 2.

Se hace programación dinámica en orden topológico (distancia creciente). Path-connection representa 'cuántos caminos óptimos pasan por esta celda', ponderados exponencialmente. La celda en más caminos óptimos tiene mayor prior.

connection[v] = Σ connection[u] · ponderación

ponderación = exp(−λ · costo(u,v))

Valores mayores (cercanos a 1) premian más los caminos cortos y valores menores hacen la distribución más uniforme.

A continuación se ilustra el DAG con dos rutas hacia dest y una arista adicional A→C, junto al cálculo detallado de connection[C]:

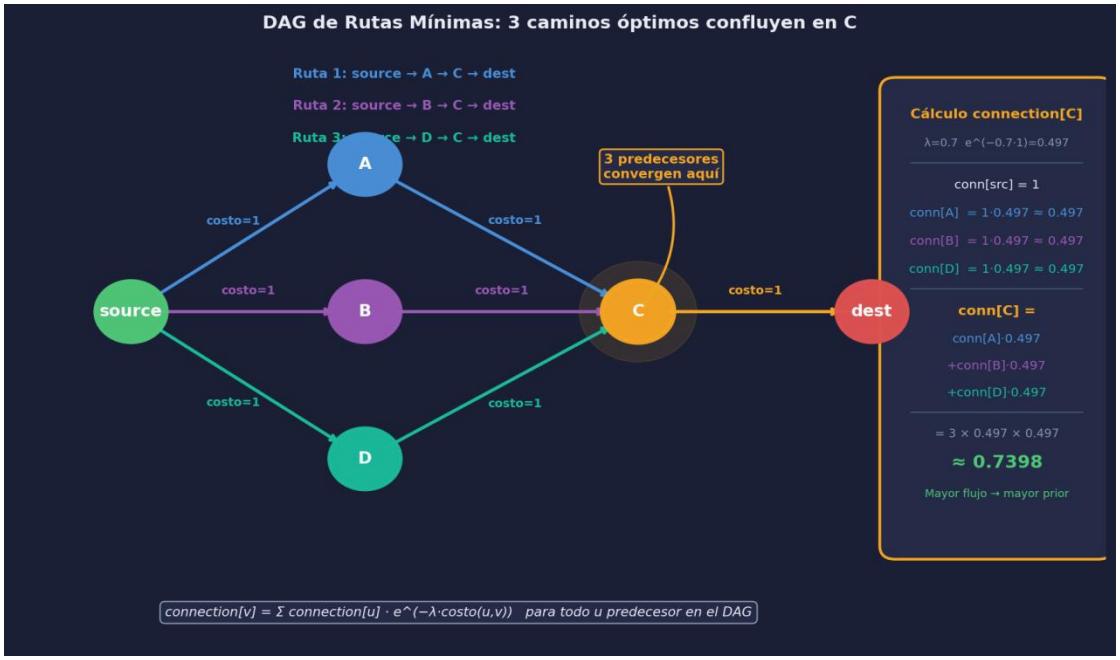

Figura 6 — DAG de rutas mínimas y cálculo de connection[C]: dos predecesores (B y A) suman sus flujos

6.2 Fórmula de Prior

```
prior(r,c) = flow_fwd_pid · flow_bwd_pid + flow_fwd_opp · flow_bwd_opp
Normalizado a distribución de probabilidad (suma = 1)
```

El fwd es conectividad desde el borde A y bwd es la conectividad desde el borde B, se calcula para el jugador y su oponente. El producto fwd*bwd tiene una interpretación directa: es el número de pares de caminos (uno desde el borde A, otro desde el borde B) que pasan simultáneamente por (r,c), es decir una conexión virtual completa. La fórmula de prior integra conectividad del jugador como ofensiva y conectividad del oponente como defensiva.

6.3 VC Boost — Amplificación de Celdas Críticas

Las celdas identificadas como portadoras de Virtual Connections (ver Sección 7) reciben un boost multiplicativo de 2.5× sobre su prior de path-connection:

```
VC_BOOST = 2.5

if vc_carriers and (r, c) in vc_carriers:
    v *= VC_BOOST
```

El boost es necesario porque path-connection mide densidad de caminos en el tablero actual, pero los VCs detectan conexiones tácticas que path-connection no ve directamente. Un carrier de bridge puede tener bajo flujo pero ser la única celda que garantiza la conexión.

7 VC Solver (Virtual Connections)

Es una promesa de conexión: el jugador puede garantizar conexión entre dos nodos independientemente de lo que juegue el oponente, siempre que el rival no pueda bloquear todos los caminos alternativos simultáneamente.

El VC más simple y más frecuente en Hex es el bridge. Se forma cuando dos fichas propias comparten exactamente dos vecinos vacíos comunes:

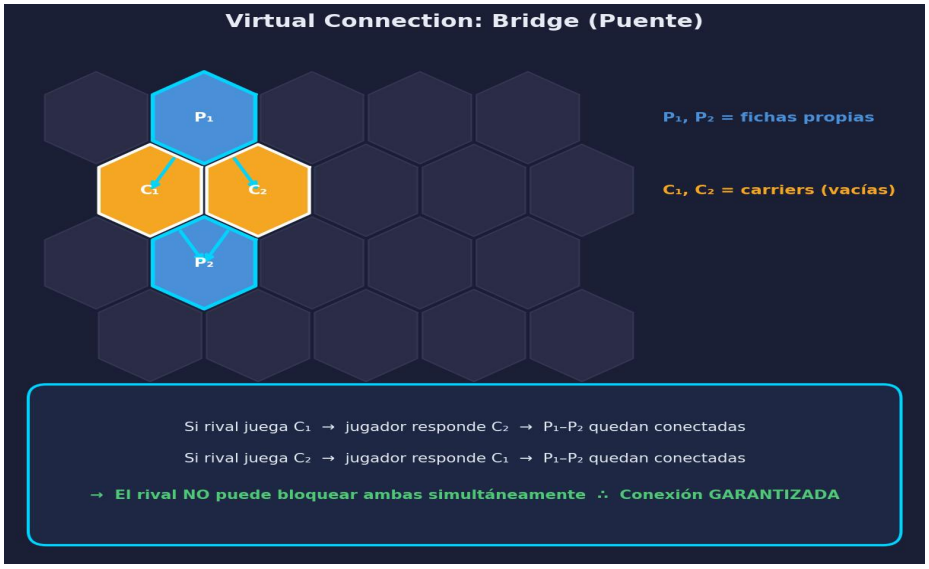

Figura 7 — Puente (Bridge): P_1 y P_2 conectadas mediante carriers C_1 y C_2. Conexión garantizada.

Conclusión: la conexión (r1,c1)–(r2,c2) está GARANTIZADA aunque el tablero esté vacío entre ellas.

7.1 Grafo VC

Los puentes se organizan en un grafo donde los nodos son grupos conexos de fichas del jugador y las aristas son puentes entre grupos distintos con el carrier (conjunto de celdas conectoras) adjunto.

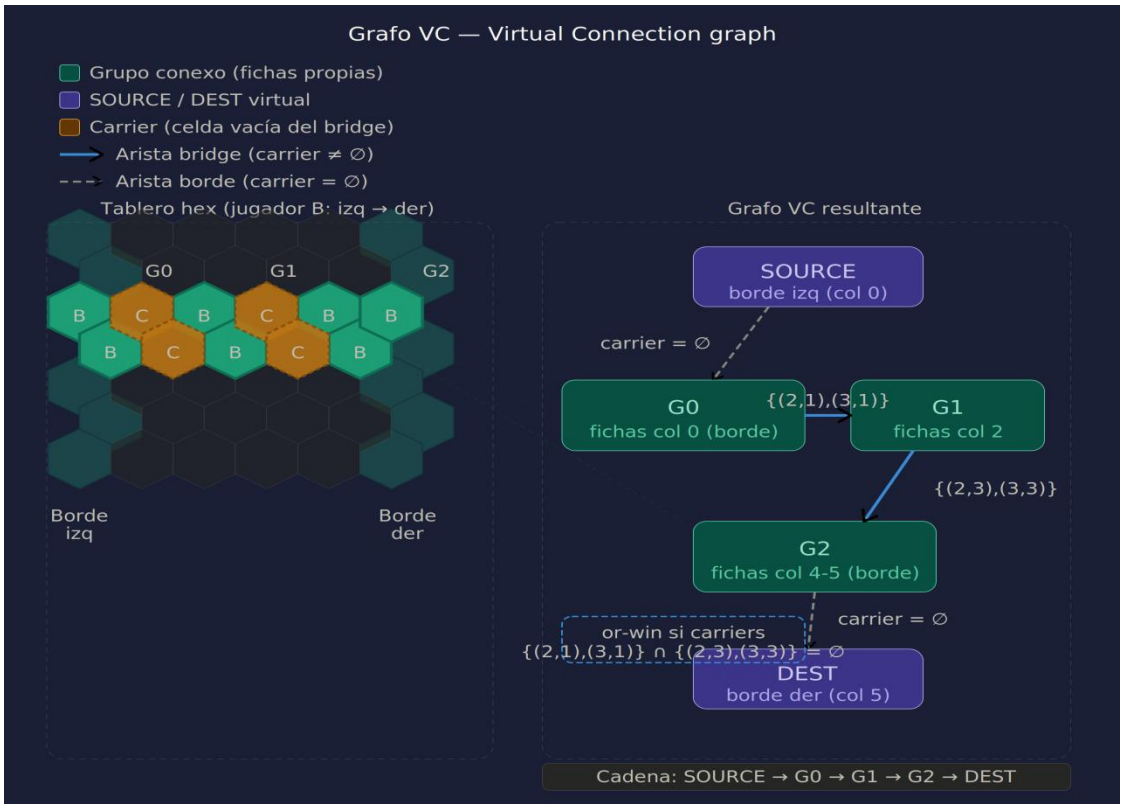

Figura 8 — Grafo VC Los bridges G0→G1 y G1→G2 tienen carriers disjuntos → _vc_or_rule_bfs confirma or-win

```
Nodos del grafo VC:
    0 .. ng-1  =  grupos conexos del jugador
    SOURCE     =  nodo virtual → borde origen
    DEST       =  nodo virtual → borde destino

SOURCE se conecta a grupos que tocan el borde origen (costo 0)
DEST   se conecta a grupos que tocan el borde destino (costo 0)

Aristas: bridge(grupo_A, grupo_B, carrier)
         carrier = frozenset de las 2 celdas vacías del bridge
```

7.2 Or-Rule

La or-rule dice: si existen dos caminos de SOURCE a DEST en el grafo VC cuyos carriers son completamente disjuntos, el jugador tiene victoria garantizada.

El algoritmo hace BFS expandiendo solo por aristas cuyos carriers son disjuntos con los ya usados. Si no hay or-win, buscamos si existe una sola celda vacía cuya eliminación desconecta todos los caminos de SOURCE a DEST en el grafo VC. Esa celda es un forced move: el jugador debe jugarla (para completar su cadena) o el rival debe jugarla (para bloquear).

Solo se activa cuando hay 6 o menos carriers en el camino mínimo (la cadena está casi completa). En otras situaciones hay demasiados candidatos y el costo no vale la pena.

8 Dead Cell Pruning

Una celda vacía es muerta si no puede ser parte de ningún camino ganador para ningún jugador. Formalmente:

```
muerta(r,c) = [inf_fp  OR  inf_bp]  AND  [inf_fo  OR  inf_bo]
inf_fp = dist_fwd_pid(r,c) es infinita.   Ambos jugadores la consideran inútil.
```

Cuando la distancia de un jugador desde el borde A hasta (r,c) es infinita, significa que ningún camino de ese jugador pasa por esa celda. Si esto ocurre para ambos bordes o para ambos jugadores, la celda es completamente irrelevante para el resultado de la partida.

En apertura (tablero casi vacío), casi ninguna celda es muerta porque todos los caminos son posibles. A medida que las fichas dividen el tablero en regiones, algunas celdas quedan encerradas en zonas que no pueden contribuir a la conexión de nadie.

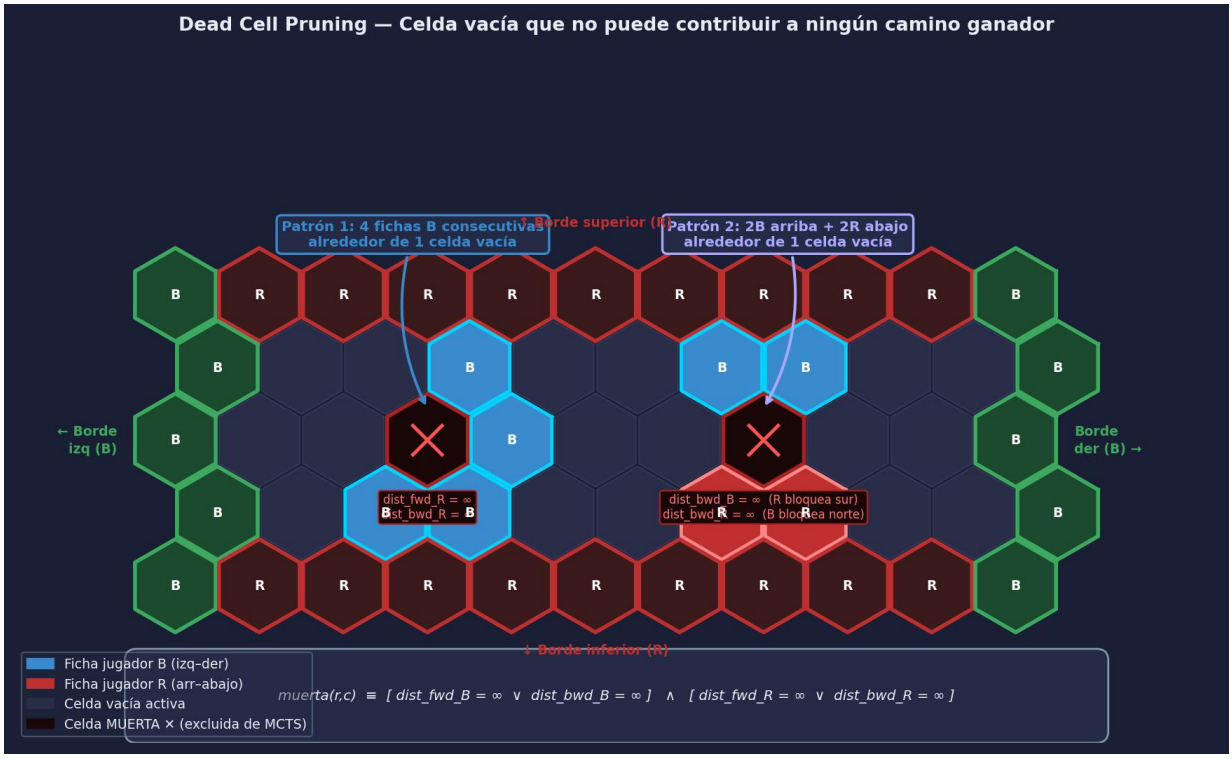

Figura 8 — Ejemplo de Dead Cell Pruning: celdas marcadas ╳ excluidas por dist = ∞

Las celdas muertas se excluyen de live_empty antes de construir el árbol MCTS. Esto tiene tres efectos:

- Menor branching factor: el árbol tiene menos hijos por nodo → más profundidad con las mismas iteraciones.
- Rollouts más cortos: hay menos celdas vacías que rellenar en cada simulación → más simulaciones por segundo.
- Costo cero: el check se hace dentro de _compute_priors, que ya calculó las distancias Dijkstra. No hay costo adicional.

9 Rollout Semi-Smart con Terminación Temprana

El rollout clásico de MCTS llena el tablero completamente con jugadas aleatorias alternadas y evalúa el ganador al final. La terminación temprana funciona con el UF: después de cada jugada, la ficha se añade al UF y se comprueba si el jugador conecta sus bordes. En Hex, el ganador se determina antes de llenar todo el tablero (típicamente a mitad). Esto reduce el rollout promedio.

9.1 Orden Semi-Smart

```
score(r,c) = prior(r,c) + max_prior · Uniform(0,1)
max_prior como escala: prior alto gana al ruido con prob ~82%
```

Con esta escala de ruido, la celda de mayor prior gana aproximadamente el 82% de las veces contra ruido puro. Las celdas de menor prior pierden contra el ruido con frecuencia, produciendo orden casi aleatorio para ellas. El resultado global: ~70% de inteligencia, ~30% de aleatoriedad.

¿Por qué no hacer el rollout completamente determinista (100% prior)? Porque un rollout determinista siempre jugaría las mismas jugadas, lo que hace que AMAF sobre-ajuste a esos patrones específicos y pierda generalidad. El ruido mantiene diversidad.

10 Flujo Completo y Jerarquía de Decisión

10.1 Caché de vecindades

Antes de comenzar el algoritmo que da vida a play() se precomputa una vez, por tamaño de tablero, el conjunto de vecinos de cada celda almacenando la información en un diccionario teniendo en cuenta que even-r layout tiene dos conjuntos de 6 direcciones posibles cada uno según la paridad de la fila.

10.2 Los Cinco Niveles

play() implementa cinco niveles de decisión en orden de prioridad decreciente. Si un nivel superior resuelve la posición, MCTS nunca se ejecuta:

1. Victoria en 1-ply: para cada celda vacía, clonar UF, colocar ficha, comprobar VL==VR o VT==VB. Si existe alguna → ganar inmediatamente.
2. Bloqueo en 1-ply: mismo proceso para el oponente. Si el rival gana en una celda → jugar ahí para bloquearlo.
3. Forced move VC propio: analizar la cadena de bridges propia. Si hay un único punto de corte → jugarlo para completar la conexión garantizada.
4. Forced move VC rival: analizar la cadena de bridges del rival. Si hay un único punto de corte → jugarlo para bloquear la conexión rival.
5. MCTS: ningún nivel anterior resolvió la posición. Ejecutar el bucle MCTS con PUCT + AMAF y path-connection hasta agotar el tiempo.